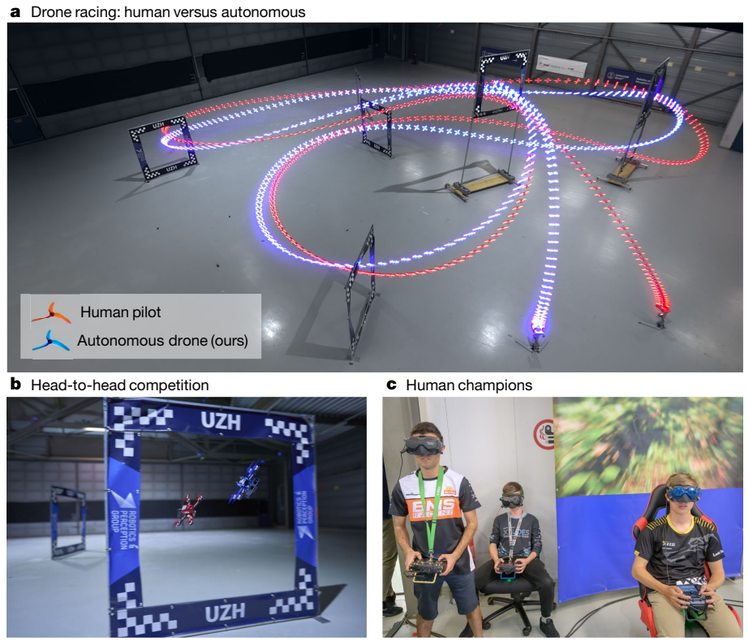


Fig. 1 | Drone racing. a, Swift (blue) races head-to-head against Alex Vanover, the 2019 Drone Racing League world champion (red). The track comprises seven square gates that must be passed in order in each lap. To win a race, a competitor has to complete three consecutive laps before its opponent. **b**, A close-up view of Swift, illuminated with blue LEDs, and a human-piloted drone, illuminated with red LEDs. The autonomous drones used in this work rely only on onboard sensory measurements, with no support from external infrastructure, such as motion-capture systems. **c**, From left to right: Thomas Bitmatta, Marvin Schaepper and Alex Vanover racing their drones through the track. Each pilot wears a headset that shows a video stream transmitted in real time from a camera aboard their aircraft. The headsets provide an immersive 'first-person-view' experience. **c**, Photo by Regina Sablotny.

(one controlled by a human pilot and one controlled by Swift) start from a podium. The race is set off by an acoustic signal. The first vehicle that completes three full laps through the track, passing all gates in the correct order in each lap, wins the race.

Swift won several races against each of the human pilots and achieved the fastest race time recorded during the events. Our work marks the first time, to our knowledge, that an autonomous mobile robot achieved world-champion-level performance in a real-world competitive sport.

The Swift system

Swift uses a combination of learning-based and traditional algorithms to map onboard sensory readings to control commands. This mapping comprises two parts: (1) an observation policy, which distils high-dimensional visual and inertial information into a task-specific low-dimensional encoding, and (2) a control policy that transforms the encoding into commands for the drone. A schematic overview of the system is shown in Fig. 2.

The observation policy consists of a visual–inertial estimator[32,33] that operates together with a gate detector[26], which is a convolutional neural network that detects the racing gates in the onboard images. Detected gates are then used to estimate the global position and orientation of the drone along the race track. This is done by a camera-resectioning algorithm[34] in combination with a map of the track. The estimate of the global pose obtained from the gate detector is then combined with the estimate from the visual–inertial estimator by means of a Kalman filter, resulting in a more accurate representation of the robot's state. The control policy, represented by a two-layer perceptron, maps the output of the Kalman filter to control commands for the aircraft. The policy is trained using on-policy model-free deep RL[31] in simulation. During training, the policy maximizes a reward that combines progress towards the next racing gate[35] with a perception objective that rewards keeping the next gate in the field of view of the camera. Seeing the next gate is rewarded because it increases the accuracy of pose estimation.

Optimizing a policy purely in simulation yields poor performance on physical hardware if the discrepancies between simulation and reality are not mitigated. The discrepancies are caused primarily by two factors: (1) the difference between simulated and real dynamics and (2) the noisy estimation of the robot's state by the observation policy when provided with real sensory data. We mitigate these discrepancies by collecting a small amount of data in the real world and using this data to increase the realism of the simulator.

Specifically, we record onboard sensory observations from the robot together with highly accurate pose estimates from a motion-capture system while the drone is racing through the track. During this data-collection phase, the robot is controlled by a policy trained in simulation that operates on the pose estimates provided by the motion-capture system. The recorded data allow to identify the characteristic failure modes of perception and dynamics observed through the race track. These intricacies of failing perception and unmodelled dynamics are dependent on the environment, platform, track and sensors. The perception and dynamics residuals are modelled using Gaussian processes[36] and *k*-nearest-neighbour regression, respectively. The motivation behind this choice is that we empirically

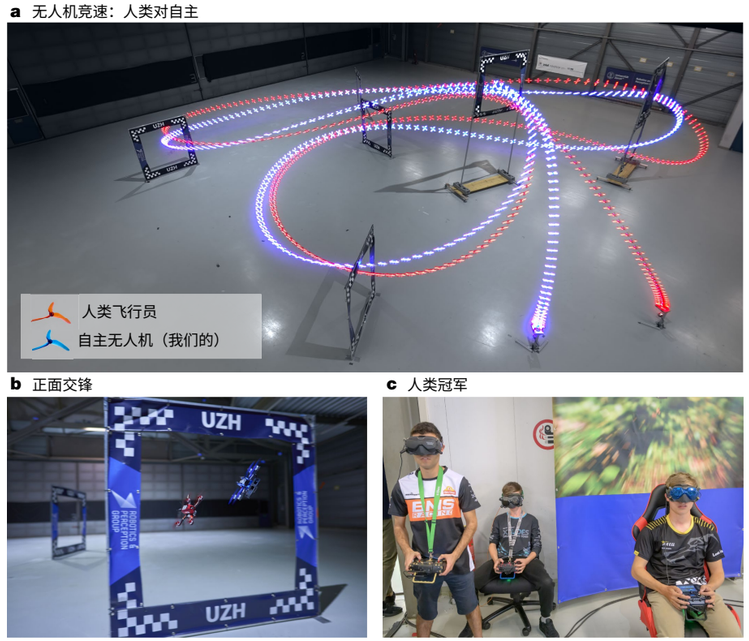


图1 | 无人机竞速。a，Swift（蓝色）与2019年无人机竞速联赛世界冠军Alex Vanover（红色）进行头对头竞速。赛道包含七个方形门，每圈必须按顺序通过。要赢得比赛，参赛者必须在对手之前完成连续三圈。**b**，Swift的近距离视图，以蓝色LED灯照明，以及人类驾驶无人机，以红色LED灯照明。本工作中使用的自主无人机仅依赖机载传感器测量，无外部基础设施（如动作捕捉系统）的支持。**c**，从左至右：Thomas Bitmatta、Marvin Schaepper和Alex Vanover驾驶他们的无人机穿过赛道。每位飞行员佩戴一个头戴式设备，显示从其飞行器上的摄像头实时传输的视频流。头戴式设备提供沉浸式的“第一人称视角”体验。**c**，摄影：Regina Sablotny。

（一架由人类飞行员控制，一架由Swift控制）从起飞台出发。比赛由声学信号启动。第一辆完成赛道上完整三圈、且每圈均按正确顺序通过所有门的车辆赢得比赛。

Swift在与每位人类飞行员的多次比赛中均获胜，并取得了赛事期间记录的最快比赛时间。据我们所知，我们的工作首次实现了自主移动机器人在真实世界竞技体育中达到世界冠军级表现。

Swift系统

Swift结合了基于学习的方法和传统算法，将机载传感读数映射为控制指令。这一映射包括两部分：（1）观测策略，将高维视觉和惯性信息提炼为任务特定的低维编码；（2）控制策略，将编码转换为无人机的指令。系统的概览示意图见图2。

观测策略由一个视觉惯性估计器组成[32,33]，它与门检测器协同运行[26]，，门检测器是一个卷积神经网络，用于检测机载图像中的赛道门。检测到的赛道门随后被用于估计无人机沿赛道的全局位置和姿态。这是通过相机重定位算法[34]结合赛道地图完成的。然后，通过卡尔曼滤波器将由门检测器获得的全局位姿估计与视觉惯性估计器的估计进行融合，从而获得更精确的表示

表示机器人状态。控制策略由两层感知机表示，将卡尔曼滤波器的输出映射为飞行器的控制指令。该策略在仿真中使用同策略无模型深度强化学习[31]进行训练。在训练期间，策略最大化一个奖励，该奖励将朝向下一赛道门的进展[35]与一个感知目标结合起来，后者奖励将下一赛道门保持在摄像头视野内。看到下一赛道门会获得奖励，因为这会提高位姿估计的准确性。

如果在仿真中纯粹优化策略，当仿真与现实之间的差异未被缓解时，在物理硬件上会产生较差的性能。这些差异主要由两个因素引起：(1) 仿真动力学与真实动力学之间的差异，以及(2) 当观测策略面对真实传感数据时，对机器人状态的噪声估计。我们通过在现实世界中收集少量数据，并使用这些数据来提高仿真器的真实感，从而缓解这些差异。

具体而言，我们记录了机器人的机载传感器观测数据，以及无人机在赛道上竞速时由动作捕捉系统提供的高精度位姿估计。在此数据收集阶段，机器人由在仿真中训练的策略控制，该策略基于动作捕捉系统提供的位姿估计进行操作。所记录的数据能够识别出在赛道上观察到的感知和动力学的典型失效模式。感知失效和未建模动力学的这些复杂特性取决于环境、平台、赛道和传感器。感知和动力学残差分别使用高斯过程[36]和*k*近邻回归进行建模。这一选择的动机是我们根据经验

Article

Methods

Quadrotor simulation

Quadrotor dynamics. To enable large-scale training, we use a high-fidelity simulation of the quadrotor dynamics. This section briefly explains the simulation. The dynamics of the vehicle can be written as

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{\mathbf{p}}_{\mathcal{WB}} \\ \dot{\mathbf{q}}_{\mathcal{WB}} \\ \dot{\mathbf{v}}_{\mathcal{W}} \\ \dot{\boldsymbol{\omega}}_{\mathcal{B}} \\ \dot{\Omega} \end{bmatrix} = \begin{bmatrix} \mathbf{v}_{\mathcal{W}} \\ \mathbf{q}_{\mathcal{WB}} \cdot \begin{bmatrix} 0 \\ \boldsymbol{\omega}_{\mathcal{B}}/2 \end{bmatrix} \\ \frac{1}{m}(\mathbf{q}_{\mathcal{WB}} \odot (\mathbf{f}_{\text{prop}} + \mathbf{f}_{\text{aero}})) + \mathbf{g}_{\mathcal{W}} \\ J^{-1}(\boldsymbol{\tau}_{\text{prop}} + \boldsymbol{\tau}_{\text{mot}} + \boldsymbol{\tau}_{\text{aero}} + \boldsymbol{\tau}_{\text{iner}}) \\ \frac{1}{k_{\text{mot}}}(\Omega_{\text{ss}} - \Omega) \end{bmatrix}, \quad (1)$$

in which $\odot$ represents quaternion rotation, $\mathbf{p}_{\mathcal{WB}}$, $\mathbf{q}_{\mathcal{WB}}$, $\mathbf{v}_{\mathcal{W}}$ and $\boldsymbol{\omega}_{\mathcal{B}}$ denote the position, attitude quaternion, inertial velocity and body rates of the quadcopter, respectively. The motor time constant is k_{mot} and the motor speeds $\boldsymbol{\Omega}$ and $\boldsymbol{\Omega}_{\text{ss}}$ are the actual and steady-state motor speeds, respectively. The matrix J is the inertia of the quadcopter and $\mathbf{g}_{\mathcal{W}}$ denotes the gravity vector. Two forces act on the quadrotor: the lift force $\mathbf{f}_{\text{prop}}$ generated by the propellers and an aerodynamic force $\mathbf{f}_{\text{aero}}$ that aggregates all other forces, such as aerodynamic drag, dynamic lift and induced drag. The torque is modelled as a sum of four components: the torque generated by the individual propeller thrusts $\boldsymbol{\tau}_{\text{prop}}$, the yaw torque $\boldsymbol{\tau}_{\text{mot}}$ generated by a change in motor speed, an aerodynamic torque $\boldsymbol{\tau}_{\text{aero}}$ that accounts for various aerodynamic effects such as blade flapping and an inertial term $\boldsymbol{\tau}_{\text{iner}}$. The individual components are given as

$$\mathbf{f}_{\text{prop}} = \sum_i \mathbf{f}_i, \ \boldsymbol{\tau}_{\text{prop}} = \sum_i \boldsymbol{\tau}_i + \mathbf{r}_{\text{P},i} \times \mathbf{f}_i, \quad (2)$$

$$\boldsymbol{\tau}_{\text{mot}} = J_{\text{m+p}} \sum_i \boldsymbol{\zeta}_i \dot{\Omega}_i, \ \boldsymbol{\tau}_{\text{iner}} = -\boldsymbol{\omega}_{\mathcal{B}} \times J\boldsymbol{\omega}_{\mathcal{B}} \quad (3)$$

in which $\mathbf{r}_{\text{P},i}$ is the location of propeller i, expressed in the body frame, and $\mathbf{f}_i$ and $\boldsymbol{\tau}_i$ are the forces and torques, respectively, generated by the ith propeller. The axis of rotation of the ith motor is denoted by $\boldsymbol{\zeta}_i$, the combined inertia of the motor and propeller is $J_{\text{m+p}}$ and the derivative of the ith motor speed is $\dot{\Omega}_i$. The individual propellers are modelled using a commonly used quadratic model, which assumes that the lift force and drag torque are proportional to the square of the propeller speed Ω_i:

$$\mathbf{f}_i(\Omega_i) = \begin{bmatrix} 0 & 0 & c_{\text{l}} \cdot \Omega_i^2 \end{bmatrix}^{\top}, \quad \boldsymbol{\tau}_i(\Omega_i) = \begin{bmatrix} 0 & 0 & c_{\text{d}} \cdot \Omega_i^2 \end{bmatrix}^{\top} \quad (4)$$

in which c_{l} and c_{d} denote the propeller lift and drag coefficients, respectively.

Aerodynamic forces and torques. The aerodynamic forces and torques are difficult to model with a first-principles approach. We thus use a data-driven model[43]. To maintain the low computational complexity required for large-scale RL training, a grey-box polynomial model is used rather than a neural network. The aerodynamic effects are assumed to primarily depend on the velocity $\mathbf{v}_{\mathcal{B}}$ (in the body frame) and the average squared motor speed $\overline{\Omega^2}$. The aerodynamic forces f_x, f_y and f_z and torques τ_x, τ_y and τ_z are estimated in the body frame. The quantities v_x, v_y and v_z denote the three axial velocity components (in the body frame) and v_{xy} denotes the speed in the (x, y) plane of the quadrotor. On the basis of insights from the underlying physical processes, linear and quadratic combinations of the individual terms are selected. For readability, the coefficients multiplying each summand have been omitted:

$$\begin{aligned}
f_x &\sim v_x + v_x|v_x| + \overline{\Omega^2} + v_x\,\overline{\Omega^2} \\
f_y &\sim v_y + v_y|v_y| + \overline{\Omega^2} + v_y\,\overline{\Omega^2} \\
f_z &\sim v_z + v_z|v_z| + v_{xy} + v_{xy}^2 + v_{xy}\,\overline{\Omega^2} + v_z\,\overline{\Omega^2} + v_{xy}\,v_z\,\overline{\Omega^2} \\
\tau_x &\sim v_y + v_y|v_y| + \overline{\Omega^2} + v_y\,\overline{\Omega^2} + v_y|v_y|\,\overline{\Omega^2} \\
\tau_y &\sim v_x + v_x|v_x| + \overline{\Omega^2} + v_x\,\overline{\Omega^2} + v_x|v_x|\,\overline{\Omega^2} \\
\tau_z &\sim v_x + v_y
\end{aligned}$$

The respective coefficients are then identified from real-world flight data, in which motion capture is used to provide ground-truth forces and torque measurements. We use data from the race track, allowing the dynamics model to fit the track. This is akin to the human pilots' training for days or weeks before the race on the specific track that they will be racing. In our case, the human pilots are given a week of practice on the same track ahead of the competition.

Betaflight low-level controller. To control the quadrotor, the neural network outputs collective thrust and body rates. This control signal is known to combine high agility with good robustness to simulation-to-reality transfer[44]. The predicted collective thrust and body rates are then processed by an onboard low-level controller that computes individual motor commands, which are subsequently translated into analogue voltage signals through an electronic speed controller (ESC) that controls the motors. On the physical vehicle, this low-level proportional–integral–derivative (PID) controller and ESC are implemented using the open-source Betaflight and BLHeli32 firmware[45]. In simulation, we use an accurate model of both the low-level controller and the motor speed controller.

Because the Betaflight PID controller has been optimized for human-piloted flight, it exhibits some peculiarities, which the simulation correctly captures: the reference for the D-term is constantly zero (pure damping), the I-term gets reset when the throttle is cut and, under motor thrust saturation, the body rate control is assigned priority (proportional downscaling of all motor signals to avoid saturation). The gains of the controller used for simulation have been identified from the detailed logs of the Betaflight controller's internal states. The simulation can predict the individual motor commands with less than 1% error.

Battery model and ESC. The low-level controller converts the individual motor commands into a pulse-width modulation (PWM) signal and sends it to the ESC, which controls the motors. Because the ESC does not perform closed-loop control of the motor speeds, the steady-state motor speed $\Omega_{i,\text{ss}}$ for a given PWM motor command cmd_i is a function of the battery voltage. Our simulation thus models the battery voltage using a grey-box battery model[46] that simulates the voltage based on instantaneous power consumption P_{mot}:

$$P_{\text{mot}} = \frac{c_{\text{d}}\Omega^3}{\eta} \quad (5)$$

The battery model[46] then simulates the battery voltage based on this power demand. Given the battery voltage U_{bat} and the individual motor command $u_{\text{cmd},i}$, we use the mapping (again omitting the coefficients multiplying each summand)

$$\Omega_{i,\text{ss}} \sim 1 + U_{\text{bat}} + \sqrt{u_{\text{cmd},i}} + u_{\text{cmd},i} + U_{\text{bat}}\sqrt{u_{\text{cmd},i}} \quad (6)$$

to calculate the corresponding steady-state motor speed $\Omega_{i,\text{ss}}$ required for the dynamics simulation in equation (1). The coefficients have been

文章

方法

四旋翼仿真

四旋翼动力学。为了实现大规模训练，我们使用了四旋翼动力学的高保真仿真。本节简要说明该仿真。飞行器的动力学可以写为

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{\mathbf{p}}_{\mathcal{WB}} \\ \dot{\mathbf{q}}_{\mathcal{WB}} \\ \dot{\mathbf{v}}_{\mathcal{W}} \\ \dot{\boldsymbol{\omega}}_{\mathcal{B}} \\ \dot{\Omega} \end{bmatrix} = \begin{bmatrix} \mathbf{v}_{\mathcal{W}} \\ \mathbf{q}_{\mathcal{WB}} \cdot \begin{bmatrix} 0 \\ \boldsymbol{\omega}_{\mathcal{B}}/2 \end{bmatrix} \\ \frac{1}{m}(\mathbf{q}_{\mathcal{WB}} \odot (\mathbf{f}_{\text{prop}} + \mathbf{f}_{\text{aero}})) + \mathbf{g}_{\mathcal{W}} \\ J^{-1}(\boldsymbol{\tau}_{\text{prop}} + \boldsymbol{\tau}_{\text{mot}} + \boldsymbol{\tau}_{\text{aero}} + \boldsymbol{\tau}_{\text{iner}}) \\ \frac{1}{k_{\text{mot}}}(\Omega_{\text{ss}} - \Omega) \end{bmatrix}, \quad (1)$$

其中$\odot$表示四元数旋转，$\mathbf{p}$ $\mathbf{q}$ $\mathbf{v}_{\mathcal{WB}}$ $_{\mathcal{WB}}$ $_{\mathcal{W}}$，，和$\boldsymbol{\omega}_{\mathcal{B}}$分别表示四旋翼的位置、姿态四元数、惯性速度和机体角速度。电机时间常数为k_{mot}，电机转速$\boldsymbol{\Omega}$和$\boldsymbol{\Omega}_{\text{ss}}$分别为实际电机转速和稳态电机转速。矩阵$J$为四旋翼的惯性，$_{\mathcal{W}}\mathbf{g}$表示重力向量。两个力作用于四旋翼：由螺旋桨产生的升力$\mathbf{f}_{\text{prop}}$以及气动力$\mathbf{f}_{\text{aero}}$，后者综合了所有其他力，例如气动阻力、动态升力和诱导阻力。力矩被建模为四个分量之和：各个螺旋桨推力产生的力矩$\boldsymbol{\tau}_{\text{prop}}$、电机转速变化产生的偏航力矩$\boldsymbol{\tau}_{\text{mot}}$、考虑桨叶挥舞等各种气动效应的气动力矩$\boldsymbol{\tau}_{\text{aero}}$以及惯性项$\boldsymbol{\tau}_{\text{iner}}$。各分量分别为

$$\mathbf{f}_{\text{prop}} = \sum_i \mathbf{f}_i, \ \boldsymbol{\tau}_{\text{prop}} = \sum_i \boldsymbol{\tau}_i + \mathbf{r}_{\text{P},i} \times \mathbf{f}_i, \quad (2)$$

$$\boldsymbol{\tau}_{\text{mot}} = J_{\text{m+p}} \sum_i \boldsymbol{\zeta}_i \dot{\Omega}_i, \ \boldsymbol{\tau}_{\text{iner}} = -\boldsymbol{\omega}_{\mathcal{B}} \times J\boldsymbol{\omega}_{\mathcal{B}} \quad (3)$$

其中$\mathbf{r}_{\text{P},i}$是螺旋桨i在机体坐标系中的位置，$\mathbf{f}_i$和$\boldsymbol{\tau}_i$分别是第i个螺旋桨产生的力和力矩。第i个电机的旋转轴记为$\boldsymbol{\zeta}_i$，电机和螺旋桨的合成惯性为$J_{\text{m+p}}$，第i个电机转速的导数为$\dot{\Omega}_i$。各螺旋桨采用常用的二次模型进行建模，该模型假设升力和阻力力矩与螺旋桨转速Ω_i的平方成正比：

$$\mathbf{f}_i(\Omega_i) = \begin{bmatrix} 0 & 0 & c_{\text{l}} \cdot \Omega_i^2 \end{bmatrix}^{\top}, \quad \boldsymbol{\tau}_i(\Omega_i) = \begin{bmatrix} 0 & 0 & c_{\text{d}} \cdot \Omega_i^2 \end{bmatrix}^{\top} \quad (4)$$

其中c_{l}和c_{d}分别表示螺旋桨升力系数和阻力系数。

气动力与力矩。气动力与力矩难以用第一性原理方法建模。因此，我们使用数据驱动模型[43]。为保持大规模强化学习训练所需的低计算复杂度，我们采用灰盒多项式模型，而非神经网络。假设气动效应主要取决于速度$\mathbf{v}_{\mathcal{B}}$（在机体坐标系中）以及平均电机转速平方$\overline{\Omega^2}$。气动力f_x、f_y和f_z以及力矩τ_x、τ_y和τ_z均在机体坐标系中估计。量v_x、v_y和v_z表示三个轴向速度分量（在机体坐标系中），而v_{xy}表示四旋翼在(x, y)平面内的速度。基于对底层物理过程的洞察，选取各项的线性与二次组合。为便于阅读，乘以各求和项的系数已省略：

$$\begin{aligned}
f_x &\sim v_x + v_x|v_x| + \overline{\Omega^2} + v_x\,\overline{\Omega^2} \\
f_y &\sim v_y + v_y|v_y| + \overline{\Omega^2} + v_y\,\overline{\Omega^2} \\
f_z &\sim v_z + v_z|v_z| + v_{xy} + v_{xy}^2 + v_{xy}\,\overline{\Omega^2} + v_z\,\overline{\Omega^2} + v_{xy}\,v_z\,\overline{\Omega^2} \\
\tau_x &\sim v_y + v_y|v_y| + \overline{\Omega^2} + v_y\,\overline{\Omega^2} + v_y|v_y|\,\overline{\Omega^2} \\
\tau_y &\sim v_x + v_x|v_x| + \overline{\Omega^2} + v_x\,\overline{\Omega^2} + v_x|v_x|\,\overline{\Omega^2} \\
\tau_z &\sim v_x + v_y
\end{aligned}$$

随后，从真实飞行数据中辨识各自的系数，其中利用动作捕捉提供真值力与力矩测量。我们使用赛道数据，使动力学模型能够拟合该赛道。这类似于人类飞行员在比赛前于其将要参赛的特定赛道上进行数天或数周的训练。在我们的情况中，人类飞行员在比赛前于同一赛道上获得了一周的练习时间。

Betaflight 底层控制器。为控制四旋翼，神经网络输出总推力和机体角速度。已知该控制信号兼具高敏捷性与良好的仿真到现实迁移鲁棒性[44]。预测的总推力和机体角速度随后由机载底层控制器处理，计算出各电机指令，再通过控制电机的电子调速器(ESC)转换为模拟电压信号。在实体飞行器上，该底层比例-积分-微分(PID)控制器和电子调速器使用开源 Betaflight 和 BLHeli32 固件实现[45]。在仿真中，我们使用底层控制器和电机转速控制器的精确模型。

由于Betaflight PID控制器针对人工驾驶飞行进行了优化，它表现出一些特殊性，仿真正确地捕捉到了这些特性：D项的参考值恒为零（纯阻尼），油门切断时I项会被重置，并且在电机推力饱和情况下，机体角速率控制被赋予优先级（对所有电机信号进行比例降幅以避免饱和）。用于仿真的控制器增益已从Betaflight控制器内部状态的详细日志中辨识得到。该仿真能够以低于1%的误差预测各个电机指令。

电池模型与电子调速器。底层控制器将各个电机指令转换为脉宽调制（PWM）信号，并将其发送给电子调速器，由电子调速器控制电机。由于电子调速器不对电机转速执行闭环控制，因此对于给定的PWM电机指令cmd_i，稳态电机转速$\Omega_{i,\text{ ss}}$是电池电压的函数。因此，我们的仿真使用灰箱电池模型对电池电压进行建模[46]，该模型基于瞬时功耗P_{mot}来模拟电压：

$$P_{\text{mot}} = \frac{c_{\text{d}}\Omega^3}{\eta} \quad (5)$$

电池模型[46]然后根据该功率需求模拟电池电压。给定电池电压U_{bat}和各个电机指令$u_{\text{cmd},i}$，我们使用映射（再次省略乘以每个加数的系数）

$$\Omega_{i,\text{ss}} \sim 1 + U_{\text{bat}} + \sqrt{u_{\text{cmd},i}} + u_{\text{cmd},i} + U_{\text{bat}}\sqrt{u_{\text{cmd},i}} \quad (6)$$

以计算对应稳态电机转速$\Omega_{i,\text{ ss}}$该转速为方程（1）中动力学仿真所需。系数已

Extended Data Table 1 | Network parameters and detailed metrics show how our Swift system compares with other approaches

a

Hyperparameter	Value
γ (discount factor)	0.99
ε (importance ratio clipping)	0.2
λ_1	1.0
λ_2	0.02
λ_3	-10.0
λ_4	-2e-4
λ_5	-1e-4

b

Approach	Lap time in seconds	Gate margin in meters	Completion in %
Zero-Shot Transfer [33]	4.92	0.57 — 0.41	42
Domain Randomization	1	0.34 — 0.23	19
Time-Opt. Traj. + MPC [26]	1	0.51 — 0.41	19
Ours	5.26	0.56 — 0.44	100

c

		Ground-truth observations			Noisy observations		
	Approach	Lap time in seconds	Gate margin in meters	Completion in %	Lap time in seconds	Gate margin in meters	Completion in %
Idealized dyn.	Zero-Shot Transfer [33]	4.88	0.63 — 0.46	100	1	n/a — n/a	0
	Domain Randomization	5.06	0.60 — 0.47	100	1	0.43 — 0.30	9
	Time-Opt. Traj. + MPC [26]	4.60	0.50 — 0.25	100	1	0.48 — 0.29	9
	Ours	4.88	0.63 — 0.46	100	5.26	0.56 — 0.44	100
Realistic dyn.	Zero-Shot Transfer [33]	1	0.62 — 0.62	4	1	0.41 — 0.21	9
	Domain Randomization	1	0.28 — 0.28	4	1	0.47 — 0.45	9
	Time-Opt. Traj. + MPC [26]	1	n/a — n/a	0	1	0.23 — 0.23	4
	Ours	5.20	0.51 — 0.30	100	5.42	0.48 — 0.23	100

d

	Autonomous Drone					**Vanover**					**Bitmatta**					**Schaepper**				
	Speed in m/s	Power in W	Thrust in N	Time in s	Length in m	Speed in m/s	Power in W	Thrust in N	Time in s	Length in m	Speed in m/s	Power in W	Thrust in N	Time in s	Length in m	Speed in m/s	Power in W	Thrust in N	Time in s	Length in m
Segment 1	**6.52**	**548.14**	**19.80**	**1.27**	8.27	5.62	348.91	14.96	1.42	7.98	5.93	365.78	15.87	1.36	8.07	4.92	201.06	10.47	1.59	**7.82**
Segment 2	13.72	852.20	29.01	**1.82**	**25.01**	**13.83**	**863.85**	**29.22**	1.83	25.31	13.72	854.36	28.95	1.89	25.94	10.76	508.29	19.97	2.36	25.35
Segment 3	13.70	798.69	27.60	**1.96**	**26.89**	**13.86**	**853.69**	**28.98**	2.09	28.91	13.16	727.36	25.50	2.23	29.35	12.66	636.78	23.47	2.40	30.34
Segment 4	13.43	**1047.05**	**33.69**	1.61	21.62	12.97	953.47	31.62	**1.60**	**20.72**	**13.47**	1035.91	33.39	1.67	22.53	13.11	885.53	29.95	1.77	23.20
Full Race	**13.11**	**866.48**	**29.16**	**17.46**	**228.85**	12.96	843.50	28.65	17.96	232.79	12.89	822.50	27.96	18.74	241.52	11.55	623.53	22.95	21.16	244.49

a, Training hyperparameters. **b**, Comparison with baselines that are provided with the same observation noise model used by our approach. **c**, Evaluation in simulation, with idealized dynamics (top) versus realistic dynamics (bottom) and ground-truth observations (left) versus noisy observations (right). We report the fastest achieved collision-free lap time in seconds, the average and smallest gate margin of successfully passed gates and the percentage of track completed. We compare our approach with a learning-based approach that performs zero-shot transfer, with and without domain randomization during training, as well as a traditional planning and control approach[28]. **d**, Comparison of the average speed, power, thrust, time and distance travelled for each pilot during the fastest flown race. Best numbers are indicated in bold.

扩展数据表1 | 网络参数和详细指标展示了我们的Swift系统与其他方法的比较

a

Hyperparameter	Value
γ (discount factor)	0.99
ε (importance ratio clipping)	0.2
λ_1	1.0
λ_2	0.02
λ_3	-10.0
λ_4	-2e-4
λ_5	-1e-4

b

Approach	Lap time in seconds	Gate margin in meters	Completion in %
Zero-Shot Transfer [33]	4.92	0.57 — 0.41	42
Domain Randomization	1	0.34 — 0.23	19
Time-Opt. Traj. + MPC [26]	1	0.51 — 0.41	19
Ours	5.26	0.56 — 0.44	100

c

		Ground-truth observations			Noisy observations		
	Approach	Lap time in seconds	Gate margin in meters	Completion in %	Lap time in seconds	Gate margin in meters	Completion in %
Idealized dyn.	Zero-Shot Transfer [33]	4.88	0.63 — 0.46	100	1	n/a — n/a	0
	Domain Randomization	5.06	0.60 — 0.47	100	1	0.43 — 0.30	9
	Time-Opt. Traj. + MPC [26]	4.60	0.50 — 0.25	100	1	0.48 — 0.29	9
	Ours	4.88	0.63 — 0.46	100	5.26	0.56 — 0.44	100
Realistic dyn.	Zero-Shot Transfer [33]	1	0.62 — 0.62	4	1	0.41 — 0.21	9
	Domain Randomization	1	0.28 — 0.28	4	1	0.47 — 0.45	9
	Time-Opt. Traj. + MPC [26]	1	n/a — n/a	0	1	0.23 — 0.23	4
	Ours	5.20	0.51 — 0.30	100	5.42	0.48 — 0.23	100

d

	Autonomous Drone					**Vanover**					**Bitmatta**					**Schaepper**				
	Speed in m/s	Power in W	Thrust in N	Time in s	Length in m	Speed in m/s	Power in W	Thrust in N	Time in s	Length in m	Speed in m/s	Power in W	Thrust in N	Time in s	Length in m	Speed in m/s	Power in W	Thrust in N	Time in s	Length in m
Segment 1	**6.52**	**548.14**	**19.80**	**1.27**	8.27	5.62	348.91	14.96	1.42	7.98	5.93	365.78	15.87	1.36	8.07	4.92	201.06	10.47	1.59	**7.82**
Segment 2	13.72	852.20	29.01	**1.82**	**25.01**	**13.83**	**863.85**	**29.22**	1.83	25.31	13.72	854.36	28.95	1.89	25.94	10.76	508.29	19.97	2.36	25.35
Segment 3	13.70	798.69	27.60	**1.96**	**26.89**	**13.86**	**853.69**	**28.98**	2.09	28.91	13.16	727.36	25.50	2.23	29.35	12.66	636.78	23.47	2.40	30.34
Segment 4	13.43	**1047.05**	**33.69**	1.61	21.62	12.97	953.47	31.62	**1.60**	**20.72**	**13.47**	1035.91	33.39	1.67	22.53	13.11	885.53	29.95	1.77	23.20
Full Race	**13.11**	**866.48**	**29.16**	**17.46**	**228.85**	12.96	843.50	28.65	17.96	232.79	12.89	822.50	27.96	18.74	241.52	11.55	623.53	22.95	21.16	244.49

a，训练超参数。**b**，与基线的比较，基线使用与我们方法相同的观测噪声模型。**c**，仿真评估，使用理想动力学（上）与真实动力学（下），以及真值观测（左）与噪声观测（右）。我们报告最快实现的无碰撞单圈时间（秒）、成功通过门的平均门裕度和最小门裕度，以及赛道完成度百分比。我们将我们的方法与进行零样本迁移的基于学习的方法（训练期间有和没有域随机化）以及传统的规划与控制方法进行比较[28]。**d**，比较每位飞行员在最快飞行比赛中的平均速度、功率、推力、时间和飞行距离。最佳数值以粗体表示。